Even Better Privacy

Post-quantum identity and secure messaging

William Doyle

May 22, 2026

Abstract

Even Better Privacy (EBP) is a post-quantum identity and secure communications protocol. Each *identity* is defined as a set of two public keys: one for signing and the other for encryption (via KEM). Confidentiality uses ML-KEM [1]; authentication and integrity use ML-DSA or SLH-DSA [2, 3]; bulk data are protected with AES-GCM after encapsulation. These choices follow NIST-approved post-quantum standards for the asymmetric layer. This document defines the protocol and its components.

1 Identity

EBP defines an identity as a pair of public keys: one for signing and the other for encryption.

1.1 Signing Key

Options for the signing key are:

- ML-DSA (Dilithium) [2]
- SLH-DSA (SPHINCS+) [3]

Both of these options are great for seperate purposes. Later in this paper we discuss how to use them together in a hierarchical relationship to benefit from the strengths of both.

1.2 Encryption Key

The encryption key is always ML-KEM (Kyber) [1] because that is the only NIST approved post-quantum KEM. We will consider adding other KEMs upon approval form NIST.

1.3 Supported Parameter Sets

EBP supports only the highest security parameter sets for each scheme. We choose to limit the parameter sets in this way to reduce protocol complexity.

- ML-DSA (Dilithium): ML-DSA-87
- SLH-DSA (SPHINCS+): SLH-DSA-SHA2-256s
- ML-KEM (Kyber): ML-KEM-1024

1.4 Fingerprint

The fingerprint is a bech32-encoded merkle root of the two public keys:

- **Left leaf:** SHA-256(raw signing public key bytes)
- **Right leaf:** SHA-256(raw KEM public key bytes)

The merkle tree design means the fingerprint can be verified without both keys present. This is useful in bandwidth-constrained scenarios.

The bech32 prefix indicates what kind of keys are used. The prefix always begins with "ebp" and ends with the first occurence of "1". The remaining characters are always of even length and represent the key type. The first half of the characters represent the signing key type and the second half represent the KEM key type. The supported patterns include:

Signing key	KEM key	Type codes	Bech32 prefix
ML-DSA (Dilithium)	ML-KEM (Kyber)	`d + k`	`ebpdk1`
SLH-DSA (SPHINCS+)	ML-KEM (Kyber)	`s + k`	`ebpsk1`

The fingerprint is designed to be human readable, informative, and useful.

1.5 Details

Each identity has a key value map for details. These can be used to store arbitrary data about an identity.

1.6 Opaque details

Keys prefixed with opaque:: are hashed before being commited to. This allows the identity to commit to the detail without publishing the detail directly.

2 Communication procedures

EBP defines a varity of procedures suited for various models of communication.

2.1 Signed Only

In this model the sender signs the message but does not encrypt it.

2.2 Encrypted Only

In this model the sender encrypts the message for the recipient but does not sign it. The recipient cannot verify who sent the message.

2.3 Signed and Encrypted Communication Procedures

Perhaps the most important communications models, in this model the message remains confidential and the sender is authenticated.

Whenever an EBP message is signed and encrypted it is always signed first and then encrypted, with the signature being included in the encrypted payload.

2.3.1 Single Recipient

The sender performs a single ML-KEM encapsulation against the recipient's encryption public key, producing a KEM ciphertext and a fresh 32-byte shared secret. Separately, the sender signs a recipient-bound hash envelope that commits to the recipient's fingerprint, a digest of the plaintext, and a random salt. The plaintext and the signature are packed together into a small JSON inner payload, which is then encrypted under the KEM-derived AES-256-GCM key using a fresh 12-byte random nonce. The transmitted ciphertext is the concatenation of the KEM ciphertext, the AES-GCM nonce, and the authenticated symmetric ciphertext.

2.3.2 Multiple Recipients

When more than one recipient is involved, encrypting the entire body once per recipient would be wasteful, since both the message body and any attachments would grow linearly in the number of recipients. EBP avoids this cost with a two-layer construction. The sender first generates a fresh random 32-byte AES-256 content key K and encrypts the message body, together with any attachments, exactly once under K using AES-256-GCM. To deliver K to each recipient, the sender then runs an independent ML-KEM encapsulation against each recipient's encryption public key; the shared secret produced by each encapsulation is used to AES-GCM-wrap K, yielding a small per-recipient record consisting of the recipient fingerprint, the KEM ciphertext, a wrap nonce, and the wrapped content key. The body ciphertext therefore has constant size regardless of recipient count, and each additional recipient contributes only the fixed overhead of one KEM ciphertext plus one wrapped-key record.

This wrapping step is necessary because ML-KEM is a key-encapsulation mechanism rather than a key-transport scheme: encapsulation produces a random shared secret determined by the sender's fresh randomness and the recipient's public key, so it cannot be used to directly deliver a chosen content key. Wrapping K under each KEM-derived shared secret is the bridge that lets every recipient recover the same K while preserving recipient-specific KEM confidentiality.

Authentication is performed once, not per recipient. The sender signs a canonical envelope that binds the message content, the sorted set of recipient fingerprints, and the sorted attachment manifest together with their content hashes. Including the recipient set in the signed envelope preserves the anti-surreptitious-forwarding property of the single-recipient mode: a recipient cannot drop or substitute another recipient, nor replay the signed payload to a different audience, without invalidating the signature. The signature, the recipient list, and the attachment manifest are packed alongside the plaintext into the inner JSON before symmetric encryption, so the signature itself remains confidential to the recipient set and is additionally protected by the AES-GCM authentication tag in transit.

3 References

References

[1] National Institute of Standards and Technology, *Module-Lattice-Based Key-Encapsulation Mechanism Standard (FIPS 203)*, U.S. Department of Commerce, 2024.

[2] National Institute of Standards and Technology, *Module-Lattice-Based Digital Signature Standard (FIPS 204)*, U.S. Department of Commerce, 2024.

[3] National Institute of Standards and Technology, *Stateless Hash-Based Digital Signature Standard (FIPS 205)*, U.S. Department of Commerce, 2024.